Integrated Data Pipeline

Prepared with AI Aid.

Agenda — Part 4: Integrated Data Pipeline

01 **What is a Data Pipeline?**
Concept, architecture, and why it matters

02 **Pipeline Architecture Overview**
Components: DB, API, Web + Storage

03 **Class: DataCollectionPipeline**
Constructor, __init__, class design

04 **_create_tables() Method**
DB schema for api_data, scraped_data, logs

05 **collect_from_database()**
Reading from source SQLite databases

06 **collect_from_api()**
REST API calls with session and persistence

07 **collect_from_web()**
CSS-selector-based scraping to DB

08 **_log_collection() & get_collection_stats()**
Monitoring and analytics

09 **export_all_data() & close()**
CSV export and cleanup

10 **End-to-End Usage Example**
Running the complete pipeline

What is a Data Pipeline?

Connecting data sources to usable insights

A data pipeline is a series of automated steps that:

1 — COLLECT
Fetch data from databases, APIs, websites

2 — STORE
Save to structured database tables

3 — LOG
Record what happened and any errors

4 — EXPORT
Output data in usable formats (CSV, etc.)

Real-World Analogy

Think of a pipeline like a water treatment plant:

- Raw water (dirty data) enters from rivers (APIs, websites, databases)
- It is filtered and cleaned (validation, error handling)

💡 **Pro Tip:** Always check if an API already exists before building a web scraper! APIs are more reliable, faster, and have better rate limits. Web scraping should be a last resort.

`Imports needed: sqlite3 | requests | bs4 | pandas | logging | datetime | time | json`

Pipeline Architecture Overview

How all components connect together

SQLite Database

REST API

Website (HTML)

DataCollectionPipeline Class

- `collect_from_database()`
- `collect_from_api()`
- `collect_from_web()`
- `_log_collection()`
- `get_collection_stats()`
- `export_all_data()`

SQLite DB (collected_data.db)

- `api_data`
- `scraped_data`
- `pipeline_logs`

`api_data.csv`

`scraped_data.csv`

`pipeline_logs.csv`

exports/ folder

Class Definition & Required Imports

Setting up the DataCollectionPipeline

CODE

```
import sqlite3        # Built-in DB interface
import requests       # HTTP library for APIs
from bs4 import BeautifulSoup  # HTML parser
import pandas as pd   # Data manipulation
import logging        # Activity logging
from datetime import datetime
import time           # For delays/rate limiting
import json           # JSON encode/decode

class DataCollectionPipeline:
    """
    Unified data collection from multiple
    sources: databases, APIs, web.
    """
```

📦 **Installation:** pip install requests beautifulsoup4 lxml pandas
(sqlite3, logging, json are built-in — no install needed!)

__init__() Constructor — Part 1: Logging Setup

Initializing the pipeline with database and logging

CODE

```
def __init__(self, db_path="collected_data.db"):
    """
    Initialize pipeline with database and logging.

    Args:
        db_path (str): Path to SQLite database file.
                       Default: "collected_data.db"
    """
    # ── LOGGING SETUP ──────────────────────────
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s - %(name)s - %(levelname)s"
               " - %(message)s",
        handlers=[
            logging.FileHandler("pipeline.log"),
            logging.StreamHandler()
        ]
    )
    self.logger = logging.getLogger(__name__)
```

Logging Format Breakdown

```
%(asctime)s   → timestamp

%(name)s → logger name

%(levelname)s → INFO/WARNING/ERROR

%(message)s → your text

level=logging.INFO → records INFO,
WARNING, ERROR

FileHandler → writes pipeline.log.

StreamHandler → prints to console.
```

__init__() Constructor — Part 2: DB & Session Setup

Database connection and HTTP session initialization

CODE

```
    # ── DATABASE SETUP ──────────────────────────
    self.db_path = db_path
    self.conn = sqlite3.connect(db_path)
    self._create_tables()   # Build schema

    # ── HTTP SESSION SETUP ──────────────────────
    self.session = requests.Session()
    self.session.headers.update({
        "User-Agent": "DataPipeline/1.0 (Educational)"
    })

    self.logger.info("Pipeline initialized")
```

Why use requests.Session()?

✓ Keeps TCP connection alive (faster repeated calls)

✓ Stores cookies across requests automatically

✓ Shared headers set once for all requests

✓ Better performance than requests.get() each time

User-Agent Header Explained

User-Agent tells websites WHO is visiting. Without it, many sites block "bot-like" requests. Setting a descriptive value is polite and avoids accidental bans.

sqlite3.connect() Explained

sqlite3.connect("collected_data.db")
→ Creates the file if it doesn't exist, opens it if it does.
→ Returns a Connection object stored as self.conn.

_create_tables() — Database Schema Design

Creating the 3 storage tables for collected data

CODE

```python
def _create_tables(self):
    cursor = self.conn.cursor()

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS api_data (
        id           INTEGER PRIMARY KEY AUTOINCREMENT,
        source       TEXT NOT NULL,
        data_type    TEXT NOT NULL,
        content      TEXT NOT NULL,
        collected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )""")

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS scraped_data (
        id         INTEGER PRIMARY KEY AUTOINCREMENT,
        url        TEXT NOT NULL,
        title      TEXT,
        content    TEXT,
        scraped_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )""")
```

Schema Visual

api_data
id (PK)
source
data_type
content
collected_at

scraped_data
id (PK)
url
title
content
scraped_at

pipeline_logs
id (PK)
source_type
records_collected
status
error_message
timestamp

_create_tables() — Database Schema Design

Creating the 3 storage tables for collected data

CODE

```
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS pipeline_logs (
        id                INTEGER PRIMARY KEY
AUTOINCREMENT,
        source_type       TEXT NOT NULL,
        records_collected INTEGER,
        status            TEXT,
        error_message     TEXT,
        timestamp         TIMESTAMP DEFAULT
CURRENT_TIMESTAMP
    )""")

    self.conn.commit()  # Save all changes
```

💡 **IF NOT EXISTS:** Prevents errors if tables already exist. Safe to call _create_tables() multiple times — it's idempotent.

Schema Visual

api_data
id (PK)
source
data_type
content
collected_at

scraped_data
id (PK)
url
title
content
scraped_at

pipeline_logs
id (PK)
source_type
records_collected
status
error_message
timestamp

collect_from_database() Method

Reading data from another SQLite database into a DataFrame

CODE

```
def collect_from_database(self, query, source_db_path):
    """
    Collect data from another SQLite database.

    Args:
        query (str): SQL SELECT query to run
        source_db_path (str): Path to the source DB file

    Returns:
        pd.DataFrame: Query results, or empty DataFrame on
error
    """
```

Arguments & Return Values

`query` *(str)*

Any valid SQL SELECT. Example:
"SELECT * FROM books WHERE copies_available > 0"

`source_db_path` *(str)*

Path to a different .db file
(not the pipeline's own DB)

`Returns` *(DataFrame)*

pandas DataFrame with rows/cols matching the query, or pd.DataFrame() (empty) on error.

collect_from_database() Method

Reading data from another SQLite database into a DataFrame

CODE

```
    self.logger.info(f"Collecting from: {source_db_path}")
    try:
        source_conn = sqlite3.connect(source_db_path)
        df = pd.read_sql_query(query, source_conn)
        source_conn.close()     # Always close!
        self.logger.info(f" Collected {len(df)} records")
        self._log_collection("database", len(df), "success")
        return df
    except Exception as e:
        self.logger.error(f" Database error: {e}")
        self._log_collection("database", 0, "error", str(e))
        return pd.DataFrame()   # Empty DF on failure
```

Arguments & Return Values

query *(str)*

Any valid SQL SELECT. Example:
"SELECT * FROM books WHERE copies_available > 0"

source_db_path *(str)*

Path to a different .db file
(not the pipeline's own DB)

Returns *(DataFrame)*

pandas DataFrame with rows/cols matching the query, or pd.DataFrame() (empty) on error.

Usage Example

CODE

```
library_data = pipeline.collect_from_database(
    "SELECT * FROM books WHERE copies_available > 0",
    "library.db"
)   # Returns a pandas DataFrame with matching rows
```

collect_from_api() Method

Fetching JSON data from REST APIs and persisting to DB

CODE

```
def collect_from_api(self, url, params=None):
    """
    Collect data from REST API.

    Args:
        url    (str):  API endpoint URL
        params (dict): Optional query parameters
                       e.g. {"per_page": 10, "page": 1}

    Returns:
        dict | list | None: Parsed JSON response,
                            or None on error.
    """
```

CODE

```
# Example usage:
data = pipeline.collect_from_api(
    "https://api.github.com/repos/pandas-
dev/pandas",
    params={"per_page": 10}
)
```

collect_from_api() Method

Fetching JSON data from REST APIs and persisting to DB

CODE

```
self.logger.info(f"Collecting from API: {url}")
try:
    response = self.session.get(url, params=params, timeout=10)
    response.raise_for_status()  # Raises on 4xx/5xx
    data = response.json()

    cursor = self.conn.cursor()
    cursor.execute(
        "INSERT INTO api_data (source, data_type, content)"
        " VALUES (?, ?, ?)",
        (url, "json", json.dumps(data))
    )
    self.conn.commit()
    self._log_collection("api", 1, "success")
    return data
except Exception as e:
    self._log_collection("api", 0, "error", str(e))
    return None
```

Key Concepts

`params=None`

Query parameters appended to URL. {"page": 2, "limit": 10} becomes ?page=2&limit=10 automatically.

`timeout=10`

Seconds to wait for server response. Without timeout, script can hang forever!

`raise_for_status()`

Raises HTTPError for 4xx (client error) or 5xx (server error) responses. Otherwise silent!

`response.json()`

Decodes JSON response body → Python dict or list. Shortcut for json.loads(response.text).

collect_from_web() Method

CSS-selector-based HTML scraping with BeautifulSoup

CODE

```
def collect_from_web(self, url, selectors):
    """
    Scrape data from website using CSS selectors.

    Args:
        url       (str):  Target URL to scrape
        selectors (dict): {field_name: css_selector}
            Example:
            {
              "title": "h1",
              "price": "span.price_color",
              "desc":  "#product_description ~ p"
            }

    Returns:
        dict: {field: extracted_text} or {} on error
    """
```

CSS Selector Cheat Sheet

`h1`	Element tag
`.price_color`	CSS class (dot prefix)
`#product_description`	Element ID (hash prefix)
`div.card`	Tag with class
`h1 ~ p`	Sibling of h1
`table tr td`	Nested elements
`a[href]`	Has attribute href
`.price:first-child`	First child element

🔍 **soup.select_one(selector):** Returns the FIRST matching HTML element, or None. select() (without _one) returns ALL matches as a list. get_text(strip=True) removes whitespace from extracted text.

collect_from_web() Method

CSS-selector-based HTML scraping with BeautifulSoup

CODE

```
try:
    response = self.session.get(url, timeout=10)
    response.raise_for_status()
    soup = BeautifulSoup(response.text, "lxml")

    data = {}
    for field, selector in selectors.items():
        element = soup.select_one(selector)
        data[field] = element.get_text(strip=True)
                      if element else None

    cursor.execute(
        "INSERT INTO scraped_data (url,title,content)"
        " VALUES (?,?,?)",
        (url, data.get("title"), json.dumps(data))
    )
```

🔍 **soup.select_one(selector):** Returns the FIRST matching HTML element, or None. select() (without _one) returns ALL matches as a list. get_text(strip=True) removes whitespace from extracted text.

⚠️ **robots.txt:** Always check https://website.com/robots.txt before scraping! Disallowed paths should NOT be scraped. Use time.sleep(1) between requests. Use the pipeline responsibly.

_log_collection() & get_collection_stats()

Monitoring pipeline health and data quality

CODE

```
def _log_collection(self, source_type, records, status,
                    error_msg=None):
    """
    Log each collection attempt to pipeline_logs table.

    Args:
        source_type (str): "database", "api", or "web"
        records     (int): Number of records collected
        status      (str): "success" or "error"
        error_msg   (str): Error text (None on success)
    """
    cursor = self.conn.cursor()
    cursor.execute(
        "INSERT INTO pipeline_logs"
        "(source_type, records_collected, status,
error_message)"
        "VALUES (?, ?, ?, ?)",
        (source_type, records, status, error_msg)
    )
    self.conn.commit()
```

_log_collection() & get_collection_stats()

Monitoring pipeline health and data quality

CODE

```python
def get_collection_stats(self):
    """Returns dict with collection statistics."""
    stats = {}
    cursor = self.conn.cursor()
    cursor.execute("SELECT COUNT(*) FROM api_data")
    stats["api_records"] = cursor.fetchone()[0]
    cursor.execute("SELECT COUNT(*) FROM scraped_data")
    stats["scraped_records"] = cursor.fetchone()[0]

    stats["logs"] = pd.read_sql_query("""
        SELECT source_type,
               COUNT(*) as count,
               SUM(CASE WHEN status="success" THEN 1 ELSE 0 END)
               AS successful
        FROM pipeline_logs
        GROUP BY source_type
    """, self.conn)
    return stats
```

Stats Output Example

CODE

```
{"api_records": 3, "scraped_records": 12,
 "logs":
   source_type  count  successful
   api          3      3
   web          12     11
   database     2      2  }
```

export_all_data() & close()

Saving collected data to CSV files and cleanup

CODE

```
import os
def export_all_data(self, output_dir="exports"):
    """
    Export all collected data to CSV files.

    Args:
        output_dir (str): Folder name for CSVs.
                          Default: "exports"
                          Created if not exists.
    """
```

export_all_data() & close()

Saving collected data to CSV files and cleanup

CODE

```
os.makedirs(output_dir, exist_ok=True)
                # exist_ok=True: no error if folder
                # already exists

api_df = pd.read_sql_query(
    "SELECT * FROM api_data", self.conn)
api_df.to_csv(f"{output_dir}/api_data.csv",
              index=False)

scraped_df = pd.read_sql_query(
    "SELECT * FROM scraped_data", self.conn)
scraped_df.to_csv(f"{output_dir}/scraped_data.csv",
                  index=False)

logs_df = pd.read_sql_query(
    "SELECT * FROM pipeline_logs", self.conn)
logs_df.to_csv(f"{output_dir}/pipeline_logs.csv",
               index=False)
```

Key Function Arguments

`os.makedirs()` `exist_ok=True`

Creates directory AND any missing parents. exist_ok prevents FileExistsError if folder already exists.

`.to_csv()` `index=False`

Saves DataFrame as CSV. index=False omits the default row numbers (0, 1, 2...) from the file.

export_all_data() & close()

Saving collected data to CSV files and cleanup

CODE

```
def close(self):
    """Close the database connection. Call when done."""
    self.conn.close()
    self.logger.info("Pipeline closed")
```

`conn.close()` none

Releases the DB file lock. Without this, other programs can't access the file. Always call in finally or at end.

💡 **Best Practice:** Always call pipeline.close() in a finally block or use a context manager pattern to ensure cleanup even on errors.

📁 **Output folder:** exports/api_data.csv | exports/scraped_data.csv | exports/pipeline_logs.csv

End-to-End Usage Example — Part 1

Running the complete pipeline step by step

CODE

```
# ────────────────────────────────────────────────────────────────────────────
# STEP 1: Initialize the pipeline
# Creates: collected_data.db + pipeline.log + HTTP session
# ────────────────────────────────────────────────────────────────────────────
pipeline = DataCollectionPipeline(db_path="collected_data.db")

# ────────────────────────────────────────────────────────────────────────────
# STEP 2: Collect from database
# Reads books from our library.db that have copies available
# ────────────────────────────────────────────────────────────────────────────
library_data = pipeline.collect_from_database(
    query="SELECT * FROM books WHERE copies_available > 0",
    source_db_path="library.db"
)
print(f"Books collected: {len(library_data)}")
# Output: Books collected: 7
```

End-to-End Usage Example — Part 1

Running the complete pipeline step by step

CODE

```
# ──────────────────────────────────────────────────────────────────────────
# STEP 3: Collect from GitHub API
# Fetches repository metadata (stars, forks, language, description)
# ──────────────────────────────────────────────────────────────────────────
github_data = pipeline.collect_from_api(
    url="https://api.github.com/repos/pandas-dev/pandas"
)
print(f"Repo: {github_data["name"]} | Stars: {github_data["stargazers_count"]}")
# Output: Repo: pandas | Stars: 42315
```

What happens at each step

Step 1: Creates collected_data.db file, creates 3 tables, starts pipeline.log

Step 2: Opens library.db (read-only), runs SQL, closes it, returns DataFrame

Step 3: Sends GET to GitHub API, parses JSON, saves to api_data table

End-to-End Usage Example — Part 2

Web scraping, stats, export and cleanup

CODE

```
# STEP 4: Scrape from a website using CSS selectors
book_data = pipeline.collect_from_web(
    url="http://books.toscrape.com/catalogue/a-light-in-the-attic_1000/index.html",
    selectors={
        "title":        "h1",                # → <h1>A Light in the Attic</h1>
        "price":        ".price_color",       # → <p class="price_color">£51.77</p>
        "availability": ".availability",      # → "In stock"
        "description":  "#product_description ~ p"  # sibling <p> after div#id
    }
)
print(book_data)
# {"title":"A Light in the Attic","price":"£51.77","availability":"In stock","description":"..."}
```

End-to-End Usage Example — Part 2

Web scraping, stats, export and cleanup

CODE

```
# STEP 5: Get collection statistics
stats = pipeline.get_collection_stats()
print(f"API Records:     {stats['api_records']}")      # → 1
print(f"Scraped Records: {stats['scraped_records']}")  # → 1
print(stats["logs"])  # DataFrame showing success/fail counts

# STEP 6: Export all data to CSV files
pipeline.export_all_data(output_dir="exports")
# Creates: exports/api_data.csv, exports/scraped_data.csv, exports/pipeline_logs.csv

# STEP 7: Close the connection (always do this!)
pipeline.close()
# Logs "Pipeline closed" and releases database file lock
```

🔄 **Complete Workflow:** Init → collect_from_database() → collect_from_api() → collect_from_web() → get_collection_stats() → export_all_data() → close()

⚠️ **Common Pitfall:** Forgetting pipeline.close() after export_all_data() leaves the database locked. Always close in a finally block or use try/finally pattern.

Project Folder Structure

How to organize your pipeline project

CODE

```
my_data_pipeline/
│
├── final_project.py        ← Main pipeline code
├── pipeline.log            ← Auto-generated log
├── collected_data.db       ← Pipeline's own DB
├── library.db              ← Source DB (from Part 1)
│
├── exports/                ← CSV output folder
│   ├── api_data.csv
│   ├── scraped_data.csv
│   └── pipeline_logs.csv
│
├── .env                    ← API keys (NEVER commit!)
├── requirements.txt        ← Package list
└── README.md               ← Documentation
```

Important File Contents

.env file (never commit to git!)

CODE

```
GITHUB_TOKEN=ghp_xxxxxxxxxxxxxxxxxxxx
API_KEY=your_secret_key_here

# Load in Python: from dotenv import load_dotenv;
load_dotenv()
```

.gitignore

CODE

```
.env            # API keys - never commit
*.db            # Database files
*.log           # Log files
exports/        # Generated CSVs
__pycache__/
```